Software Requirements Specifications

AI COMIC GENERATOR

Web-Based AI-Powered Comic Strip Generation System

Internal Advisor:

SIR SAAD RAZZAQ

Project Manager:

Dr. SAAD RAZZAQ

Project Team:

ABDULLAH AKRAM(BSCS51F22S058) TEAM LEADER

LARAIB REHMAN (BSCS51F22S077)

MUHAMMAD AHMED(BSCS51F22S085)

Submission Date:

1 May 2025

_____

Project Manager's Signature 2025

Document Information

Document Title	Software Requirements Specification
Project Name	AI Comic Generator
Version	1.1
Date	December 2024
Student Names	Abdullah Akram - BSCS51F22S058 Laraib Rehman - BSCS51F22S077 Muhammad Ahmed - BSCS51F22S085
Project Type	Final Year Project
Department	Computer Science
Status	Final

Definition of Terms, Acronyms and Abbreviations

This section should provide the definitions of all terms, acronyms, and abbreviations required to interpret the terms used in the document properly.

Term	Description
NLP	Natural Language Processing
API	Application Programming Interface
UI	User Interface
PNG/PDF	Portable Network Graphics / Portable Document Format
LLM	Large Language Model
FYP	Final Year Project
DB	Database

Table of Contents

11. Project Constraints and Limitations

11.1 Technical Constraints

11.2 Budget Constraints

11.3 Time Constraints

11.4 Scope Limitations

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete description of the AI Comic Generator system. It defines the functional and non-functional requirements, system architecture, interfaces, and constraints for the development team and stakeholders.

1.2 Project Scope

The AI Comic Generator is an automated system that transforms text-based stories into multi-panel comic strips with consistent character representation, dialogue placement, and professional layout formatting.

Key Features:

- Text-to-comic conversion with scene segmentation
- Character consistency across panels via image upload
- Automated speech bubble generation and placement
- Cartoon-style image generation using AI models
- Export functionality (PNG/PDF formats)
- Web-based user interface for story input and comic customization

1.3 References

- React Documentation — https://react.dev
- Tailwind CSS Documentation — https://tailwindcss.com
- FastAPI Documentation — https://fastapi.tiangolo.com
- SQLAlchemy Documentation — https://docs.sqlalchemy.org
- Pollinations.ai API — https://pollinations.ai
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications

1.4 Document Overview

This document is organized into sections covering system description, functional requirements, non-functional requirements, system architecture, interface requirements, and constraints.

---

2. Overall Description

2.1 Product Perspective

The AI Comic Generator is a standalone web application designed as a Final Year Project. It does not replace any existing commercial product but demonstrates the practical integration of modern web technologies with generative AI services

System Context:

- Integrates with external AI APIs
- Uses Google Colab for GPU-accelerated image generation
- Employs cloud storage for generated assets
- Provides web interface accessible via modern browsers

2.2 Product Functions

The system provides the following major functions:

Feature	Description
User Registration & Login	Secure signup with email/password; login with session persistence via localStorage
Email OTP Verification	6-digit one-time password sent via Gmail SMTP; verified before account activation
Story Input	Text area allowing users to enter narrative stories up to a defined character limit
Panel Configuration	User selects 2, 4, 6, or 8 comic panels per generation
AI Comic Generation	Backend splits story into scenes; Pollinations.ai renders each scene as a comic panel image
Comic Preview	Multi-panel grid display with real-time image loading within the browser
Download	Export full comic as a single PNG file using html2canvas
Gallery (My Comics)	Save generated comics to browser localStorage; browse and reload saved comics
Animated Loading Screen	Marvel-style intro animation on first load with skip option

2.3 User Classes and Characteristics

Primary Users:

- **Content Creators:** Bloggers, social media creators needing visual content
- **Educators:** Teachers creating educational visual materials
- **Writers:** Authors visualizing story concepts and drafts
- **Comic Enthusiasts:** Hobbyists experimenting with comic creation

Technical Expertise: Basic to intermediate computer literacy required. No artistic or programming skills needed.

2.4 Operating Environment

Component	Specification
Client Browser	Google Chrome 100+, Mozilla Firefox 100+, Microsoft Edge 100+
Client OS	Windows 10/11, macOS 12+, Ubuntu 20.04+
Frontend Runtime	Node.js 18+ (development); served as static build in production
Backend Runtime	Python 3.9+
Backend Server	Uvicorn ASGI server on localhost:5000
Frontend Dev Server	React development server on localhost:3000
Database	SQLite 3 (file-based: comic_generator.db)
Network	Internet connection required for Pollinations.ai API and Gmail SMTP

2.5 Design and Implementation Constraints

- AI Dependency: Image generation depends on the availability of the third-party Pollinations.ai service. No fallback image generator is implemented in version 1.0.
- Free-tier AI: Pollinations.ai is a free API with no SLA; image quality and response times may vary.
- Single-server deployment: Backend and frontend run on the same machine in the current version.

- No password hashing: Passwords are stored in plaintext in SQLite in version 1.0 (flagged as a future improvement).
- No JWT authentication: Session state is managed via localStorage without token-based security.
- Image generation time: Each panel requires a separate HTTP call to Pollinations.ai; generation time scales with panel count.
- Browser storage limit: My Comics gallery is stored in localStorage (typically 5–10 MB limit).

2.6 Assumptions and Dependencies

Assumptions:

- Users have stable internet connectivity
- External API services remain available and maintain current pricing
- Users understand basic comic conventions
- Generated content will be used responsibly

Dependencies:

- Users have access to a modern web browser with JavaScript enabled.
- A valid Gmail account with App Password is configured on the backend for SMTP.
- The Pollinations.ai service remains publicly accessible without authentication.
- SQLite file permissions allow read/write on the server machine.
- Port 3000 and 5000 are available and not blocked by firewalls during local development.

---

3. Functional Requirements

3.1 User Authentication Module

3.1.1 User Registration (Signup)

The system shall provide a registration form with the following fields and behavior:

ID	Requirement	Description
FR-01	Full Name Input	System shall accept a full name field (minimum 2 characters, alphabetic).
FR-02	Email Input	System shall validate that the email belongs to an accepted domain (gmail.com, yahoo.com, outlook.com, hotmail.com, etc.).
FR-03	Password Strength	System shall enforce a password strength indicator and require a minimum of 8 characters.

ID	Requirement	Description
FR-04	Confirm Password	System shall validate that the confirm password field matches the password field.
FR-05	Show/Hide Password	System shall provide a toggle to show or hide password characters.
FR-06	Terms Checkbox	System shall require acceptance of terms and conditions before allowing submission.
FR-07	OTP Dispatch	On valid form submission, the system shall generate a 6-digit OTP and send it to the provided email address via SMTP.
FR-08	Temporary OTP Storage	OTP shall be stored server-side temporarily and expire after a defined period.

3.1.2 Email OTP Verification

ID	Requirement	Description
FR-09	OTP Input	System shall present a 6-digit OTP input field, accepting numeric characters only.
FR-10	Auto-format	System shall strip non-numeric characters and limit input to 6 digits in real time.
FR-11	OTP Validation	System shall call the backend /api/auth/verify-otp endpoint and create the user account on success.
FR-12	Resend OTP	System shall provide a resend option with a visible countdown timer.
FR-13	Error Feedback	System shall display a clear error message if the OTP is incorrect or expired.

3.1.3 User Login

ID	Requirement	Description
FR-14	Login Form	System shall accept email and password; prevent page reload on submit

ID	Requirement	Description
		(e.preventDefault).
FR-15	Credential Validation	Backend shall compare submitted credentials against stored user record.
FR-16	Session Storage	On successful login, the system shall store user_id, name, and email in localStorage.
FR-17	Redirect on Login	User shall be redirected to the Comic Generator page upon successful authentication.
FR-18	Error Messages	System shall display meaningful error messages for invalid credentials.
FR-19	Logout	System shall provide a logout action that clears localStorage session data.

3.2 Comic Generation Module

3.2.1 Story Input

ID	Requirement	Description
FR-20	Story Textarea	System shall provide a multi-line text area for story input with a visible character counter.
FR-21	Panel Selection	System shall present panel count options: 2, 4, 6, or 8 panels.
FR-22	Input Validation	System shall disable the Generate button if the story field is empty.
FR-23	Authentication Guard	Access to the Generator page shall be restricted to logged-in users.

3.2.2 Story Processing & Scene Splitting

ID	Requirement	Description
FR-24	Sentence Splitting	Backend shall split the input story on sentence delimiters (. ! ?) into individual sentences.

ID	Requirement	Description
FR-25	Scene Assignment	Backend shall group sentences into N equal scenes, where N is the requested panel count.
FR-26	Prompt Construction	Each scene shall be prefixed with 'comic book panel style:' to guide image generation.

3.2.3 AI Image Generation

ID	Requirement	Description
FR-27	Pollinations.ai Call	System shall URL-encode each scene prompt and construct a Pollinations.ai image request URL (1024×1024 px).
FR-28	Panel Data Return	Backend shall return an ordered list of panel objects {id, imageUrl} to the frontend.
FR-29	Loading State	Frontend shall display an animated loading indicator while awaiting backend response.
FR-30	Error Handling	System shall display a user-friendly error message if any panel fails to generate.
FR-31	Image Loading	Frontend shall display each panel image as it becomes available; broken image placeholders shall be shown on failure.

3.2.4 Comic Preview & Layout

ID	Requirement	Description
FR-32	Panel Grid Display	System shall display generated panels in a responsive grid layout matching the selected panel count.
FR-33	Step Management	Generator shall manage three states: 'input', 'generating', and 'preview'.
FR-34	Regenerate Option	User shall be able to return to input and generate a new comic from the preview step.

3.3 Download Module

ID	Requirement	Description
FR-35	HTML to Image	System shall use html2canvas to capture the comic panel grid as a canvas element.
FR-36	CORS Images	html2canvas shall be invoked with useCORS: true to allow cross-origin images from Pollinations.ai.
FR-37	PNG Export	Canvas shall be converted to a Blob and downloaded as a .png file via an anchor element.
FR-38	Image Load Wait	System shall delay capture until all panel images have fully loaded, using a setTimeout buffer.

3.4 Gallery Module (My Comics)

ID	Requirement	Description
FR-39	Save Comic	System shall serialize generated comic data (story, panel URLs, panel count) as JSON and persist it in localStorage under the key 'myComics'.
FR-40	Gallery View	My Comics page shall read from localStorage and display all saved comics in a responsive grid.
FR-41	Load Comic	User shall be able to select a saved comic and reload it in the Preview step of the Generator.
FR-42	Delete Comic	User shall be able to delete individual comics from the gallery; deletion shall update localStorage immediately.
FR-43	Empty State	If no comics are saved, the gallery shall display an appropriate empty-state message and call-to-action.

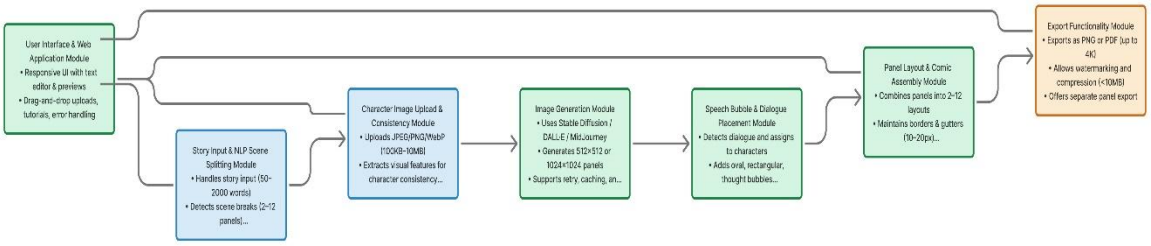
3.5 Navigation & Routing

ID	Requirement	Description

ID	Requirement	Description
FR-44	Route: /	Landing Page — accessible to all users.
FR-45	Route: /login	Login page — redirects to /generator if already logged in.
FR-46	Route: /signup	Signup page — accessible to unauthenticated users.
FR-47	Route: /verify-otp	OTP verification — receives email from signup navigation state.
FR-48	Route: /generator	Comic Generator — protected route; redirects to /login if not authenticated.
FR-49	Route: /my-comics	Gallery — protected route.
FR-50	Route: /about	About page — accessible to all users.

3.6 Loading Screen

ID	Requirement	Description
FR-51	Intro Animation	System shall display an animated loading screen on initial application load featuring flying comic images.
FR-52	Phase Transition	Loading screen shall progress through phases: image animations → logo reveal → main app.
FR-53	Skip Button	User shall be able to skip the loading animation at any time.
FR-54	Progress Tracking	System shall track animation progress and transition automatically after completion.



---

4. Non-Functional Requirements:

4.1 Performance

The system must deliver fast and efficient results. Story inputs up to 2000 words should be processed and split into scenes within **5 seconds**. Each comic panel should be generated in **under 2 minutes**, and a full 6-panel comic must be ready within **15 minutes**. The system should support **up to 10 users simultaneously** without performance drops, while the web interface should load within **3 seconds**. Database queries must execute within **500 milliseconds**.

4.2 Reliability

The system should maintain at least 95% uptime during testing and handle API errors gracefully with meaningful error messages. Failed image generations should automatically retry up to three times. To prevent data loss, an auto-save mechanism must be implemented, and all system errors should be logged for future debugging.

4.3 Usability

The platform must be user-friendly and accessible for non-technical users. A new user should be able to generate a comic within **20 minutes** of first use. The interface should require no more than **five clicks** to start comic creation and include **tooltips and navigation guidance**. All prompts and error messages should be **clear and understandable**.

4.4 Security

Security is a top priority. All user inputs must be validated to avoid **injection attacks**, and uploaded images should be scanned for **inappropriate content**. **API keys** must remain confidential and never be visible on the client side. To prevent system abuse, a **rate limit of 10**

comics per user per hour must be enforced, and user data should remain isolated in each session.

4.5 Maintainability

The codebase should follow standard conventions—**PEP 8** for Python and **ESLint** for JavaScript. Major functions must include **docstrings and comments**, and the architecture should be **modular** to enable easy updates or replacements. Configuration settings should be stored in **external environment files**, and a **README** file must provide setup and maintenance instructions.

4.6 Scalability

The system must support **horizontal scaling** of backend services to handle increased traffic. The database should be designed with **future features** in mind and allow easy integration with additional **image generation APIs**. This ensures the platform can grow without major restructuring.

4.7 Portability

The backend should run smoothly across **Windows, Linux, and macOS**, while the frontend must function on all **modern browsers**. The use of **platform-independent file paths and configurations** will ensure cross-platform compatibility and seamless deployment across environments.

---

5. System Architecture

5.1 High-Level Architecture

The system follows a three-tier architecture:

Presentation Layer (Frontend):

- React/Next.js web application
- User input forms and file upload
- Comic preview and editing interface
- Export download functionality

Application Layer (Backend):

- FastAPI/Flask REST API server
- Story parsing and scene segmentation module
- Prompt generation engine
- Image generation orchestrator
- Layout and export module

Data Layer:

- SQLite database for persistent storage
- File storage for generated images
- Cache layer for API responses

5.2 Component Diagram

Components Hierarchy:

1. **App.js**

Route definitions; wraps all pages in BrowserRouter

2. **LoadingScreen.js**

Animated intro; shown once on first load

3. **LondingPage.js**

Marketing homepage with hero, features, stats, testimonials

4. **Login.js**

Authentication form

5. **Signup.js**

Registration form with validation

6. **VerifyOTP**

6-digit OTP entry and verification

7. **ComicGenerator.js**

Core comic creation: input → generating → preview

8. **MyComics.js**

Saved comics gallery from localStorage

9. **About.js**

Project info and team cards

5.3 Data Flow

1. User submits story text and panel count via ComicGenerator.js form.
2. services/api.js sends POST /api/generate-comic with {story, panels} JSON payload.
3. Backend main.py receives the request; calls split_story_into_scenes(story, panels).
4. Story is split into sentences using regex; sentences grouped into N scenes.
5. For each scene, backend constructs a Pollinations.ai URL with URL-encoded prompt.
6. Backend returns array of {id, imageUrl} panel objects.
7. Frontend receives panels; transitions to 'preview' step; renders image grid.
8. User optionally downloads: html2canvas captures grid → Blob → PNG download.
1. User optionally saves: comic data JSON written to localStorage 'myComics' key.

6. Database Design

6.1 Overview

The system uses SQLite as its relational database, managed through SQLAlchemy ORM. The database file is named comic_generator.db and resides in the backend directory. Two tables are defined: users and comics.

6.2 Entity-Relationship Summary

One User can have zero or many Comics (one-to-many relationship). A Comic belongs to exactly one User via a foreign key on user_id.

6.3 Table: users

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Unique user identifier
name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	User's email address (used for login)
password	VARCHAR(100)	NOT NULL	User's password (plaintext in v1.0)

Column	Type	Constraints	Description
email_verified	INTEGER	DEFAULT 0	0 = unverified, 1 = verified via OTP
created_at	DATETIME	DEFAULT NOW()	Account creation timestamp

6.4 Table: comics

Column	Type	Constraints	Description
id	INTEGER	PK, Auto	Unique comic identifier
user_id	INTEGER	FK → users.id	Owner of the comic
story	TEXT	NOT NULL	Original story text submitted by the user
panels	TEXT	NOT NULL	JSON-serialized array of panel objects {id, imageUrl}
panel_count	INTEGER	NOT NULL	Number of panels selected (2, 4, 6, or 8)
created_at	DATETIME	DEFAULT NOW()	Comic creation timestamp

7. External Interface Requirements

7.1 User Interfaces

- **Design Language:** Glassmorphism with dark backgrounds (black/dark navy), backdrop blur effects, and gradient purple-to-pink accents.
- **CSS Framework:** Tailwind CSS v3 utility classes — no custom CSS files except global index.css.
- **Icons**: lucide-react icon library (Eye, Download, Sparkles, etc.).
- **Typography**: System font stack via Tailwind defaults; gradient text using bg-clip-text for headings.
- **Animations**: Tailwind transition and hover utilities; custom keyframe animations in index.css for loading screen.
- **Layout**: CSS Grid and Flexbox via Tailwind classes; responsive breakpoints at 768px and 1280px.

7.2 Software Interfaces

Interface	Version	Purpose
React.js	18+	Component-based frontend framework
React Router DOM	6+	Client-side routing
Tailwind CSS	3.3.0	Utility-first CSS framework
Axios	1+	HTTP client for API communication
html2canvas	1+	HTML-to-image capture for PNG export
lucide-react	latest	SVG icon components
FastAPI	0.100+	Python async web framework
Uvicorn	0.23+	ASGI server for FastAPI
SQLAlchemy	2+	Python ORM for SQLite
python-dotenv	1+	Environment variable management

7.3 Communication Interfaces

CI-1: Frontend-Backend Communication

- Protocol: HTTP/REST over localhost TCP.
- Frontend base URL: http://localhost:5000/api
- Data format: JSON (Content-Type: application/json).
- Timeout: 120,000 ms (2 minutes) for comic generation requests.

CI-2: Backend ↔ Pollinations.ai

- Protocol: HTTPS GET request.
- URL pattern: https://image.pollinations.ai/prompt/{encoded_prompt}?width=1024&height=1024
- Authentication: None required (public free API).
- Response: Direct image URL returned as string; frontend loads image via <img> src attribute.

CI-3: Backend ↔ Gmail SMTP

- Protocol: SMTPS (SSL) on port 465 or STARTTLS on port 587.
- Authentication: Gmail account email + App Password (not the regular account password).

- Purpose: Sending OTP verification emails to newly registered users.
- Library: Python smtplib / email.mime modules.

---

8. System Models

8.1 Use Case Diagram

Actors:

- User (Content Creator, Educator, Writer, Student, Marketer)

- System Administrator

- External APIs (OpenAI, Stability AI)

Primary Use Cases:

1. Register Account

2. Login/Logout

3. Create Comic Project

4. Input Story Text

5. Upload Character Image

6. Select Art Style

7. Generate Comic

8. Preview Comic

9. Export Comic (PNG/PDF)

10. Manage Projects (View, Edit, Delete)

11. Configure Settings

12. View Generation Progress

Administrator Use Cases:

13. Monitor System Health

14. Manage User Accounts

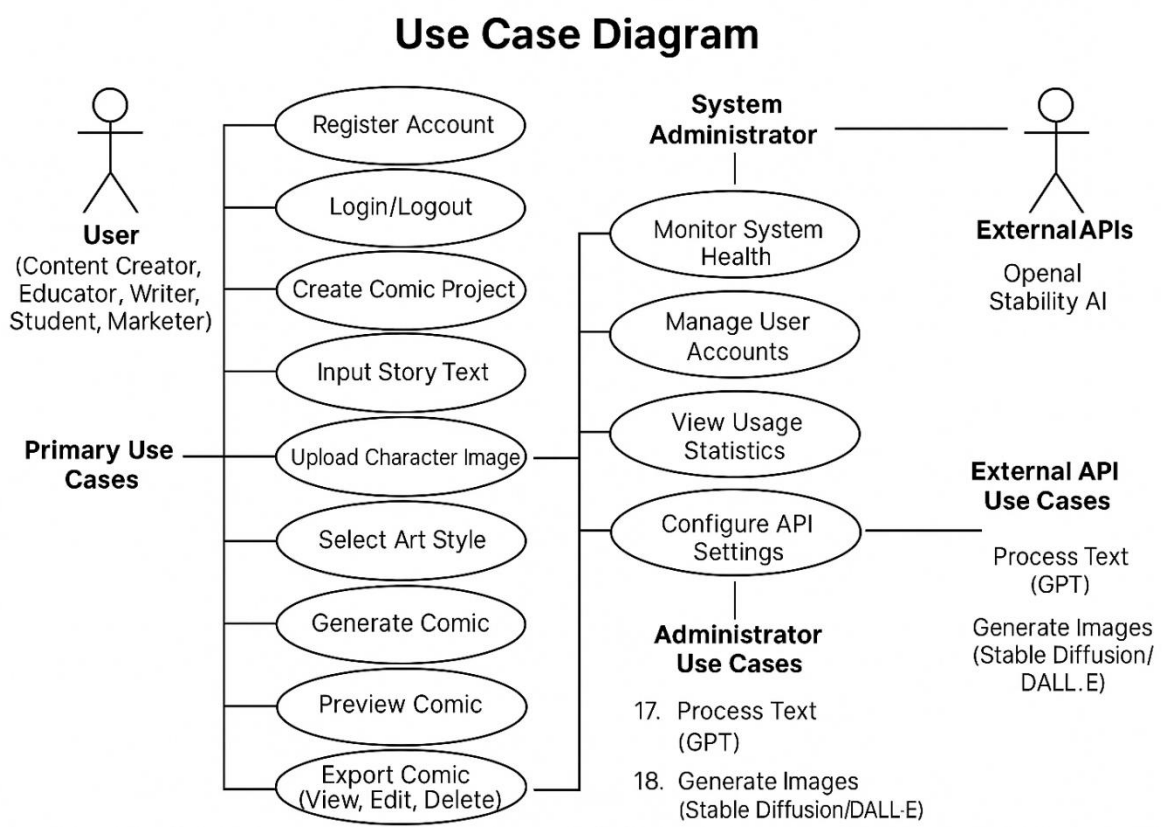
15. View Usage Statistics

16. Configure API Settings

External API Use Cases:

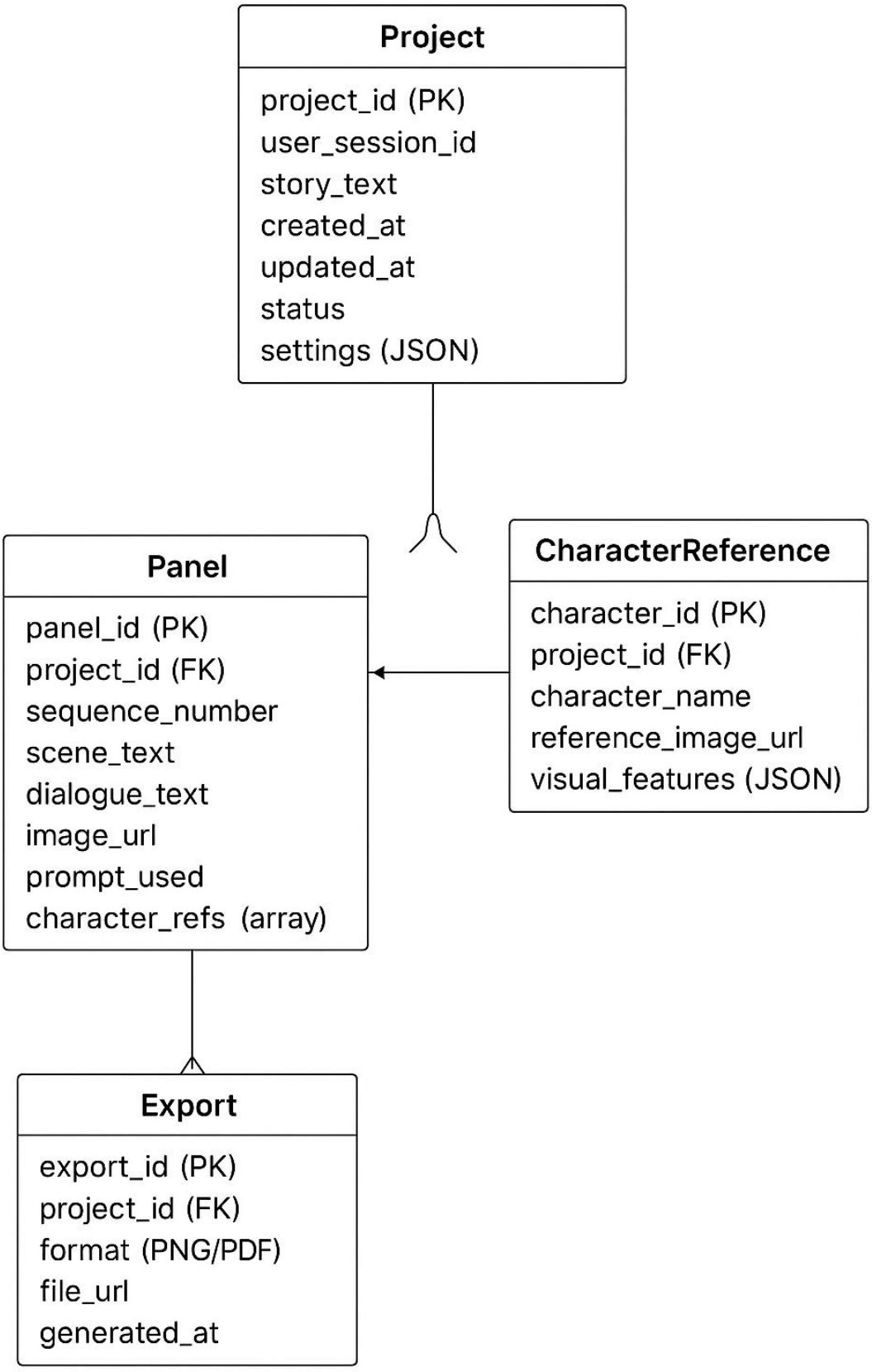
17. Process Text (GPT)

18. Generate Images (Stable Diffusion/DALL-E)



8.2 Data Model

ERD DIAGRAM


Project
project_id (PK)
user_session_id
story_text
created_at
updated_at
status
settings (JSON)
Panel
panel_id (PK)
project_id (FK)
sequence_number
scene_text
dialogue_text
image_url
prompt_used
character_refs (array)
CharacterReference
character_id (PK)
project_id (FK)
character_name
reference_image_url
visual_features (JSON)
Export
export_id (PK)
project_id (FK)
format (PNG/PDF)
file_url
generated_at

9. Acceptance Criteria

8.1 Functional Acceptance Criteria

AC-1: System successfully segments 90% of test stories into logical scenes.

AC-2: Generated images match the art style selected by user.

AC-3: Character appearance maintains 70% visual consistency across panels.

AC-4: Speech bubbles are readable and properly positioned in 85% of cases.

AC-5: Exported comics maintain resolution and layout integrity.

AC-6: Users can complete the entire workflow from input to export without errors.

8.2 Performance Acceptance Criteria

AC-7: 6-panel comic generation completes within 15 minutes.

AC-8: Web interface loads within 3 seconds.

AC-9: System handles 10 concurrent users without degradation.

8.3 Usability Acceptance Criteria

AC-10: 80% of test users successfully create a comic on first attempt.

AC-11: Users rate interface ease-of-use at 4/5 or higher.

AC-12: Users require less than 5 minutes to understand the workflow.

---

10. Testing Requirements

10.1 Unit Testing

- Test individual modules (scene splitter, prompt generator, bubble renderer)
- Achieve 80% code coverage
- Automated tests for API integrations

10.2 Integration Testing

- Test end-to-end workflow

- Verify API communication
- Database operations testing

10.3 User Acceptance Testing

- Conduct testing with 5-10 users
- Collect feedback on usability and output quality
- Measure success against acceptance criteria

10.4 Performance Testing

- Load testing with concurrent users
- API response time measurement
- Memory and resource usage monitoring

---

11. Project Constraints and Limitations

11.1 Technical Constraints

- Dependent on third-party API availability and rate limits
- GPU access required for optimal performance
- Internet connectivity mandatory
- Limited character consistency compared to fine-tuned models

11.2 Budget Constraints

- API costs may limit number of test generations
- Cloud hosting costs for deployment

11.3 Time Constraints

- 18-week development timeline
- Limited time for advanced feature implementation

11.4 Scope Limitations

- No real-time collaborative editing
- Basic character consistency only

- Limited to 12 panels per comic
- No advanced copyright management

---

11. Appendices

Appendix A: Glossary

- **Panel:** Single frame in a comic strip
- **Speech Bubble:** Graphical element containing character dialogue
- **Scene Segmentation:** Process of dividing story into panels
- **Character Consistency:** Maintaining visual appearance across panels
- **Prompt Engineering:** Crafting effective text prompts for image generation

Appendix B: Change Log

- Version 1.1 (May 1, 2026)

Appendix C: Approval Signatures

- Project Supervisor: _____
- Team Lead: _____
- Date: _____